

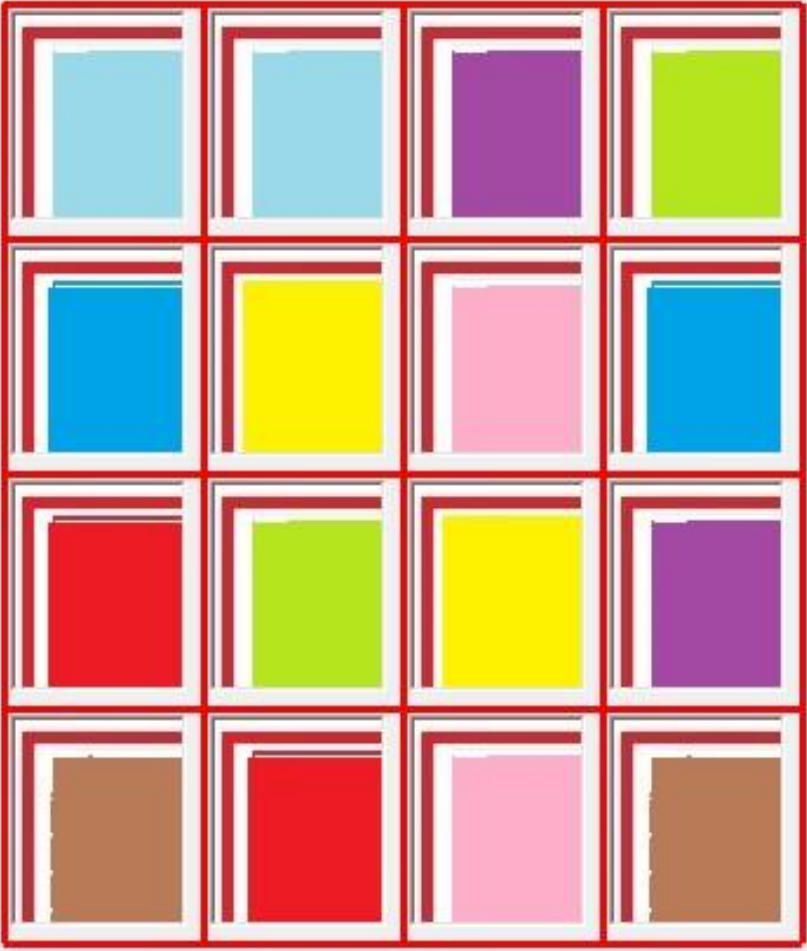
Парные картинки

Игра «парные картинки»

- Правила игры **Парные картинки** следующие: игровое поле разделено на клетки, за каждой из которых скрыта картинка. Каждая картинка имеет пару — точно такую же картинку. В начале игры
- По нажатию кнопки Start открываются ненадолго все картинки и начинается отсчет времени
- все клетки закрыты. Щелчок мыши в клетке открывает картинку. Второй щелчок — вторую картинку. Если картинки совпали, начисляется одно очко, если не совпали — обе через некоторое время исчезают. На экране остаются только парные картинки
- Чтобы победить, нужно уложиться в отведенное время

Начало работы программы

Form1



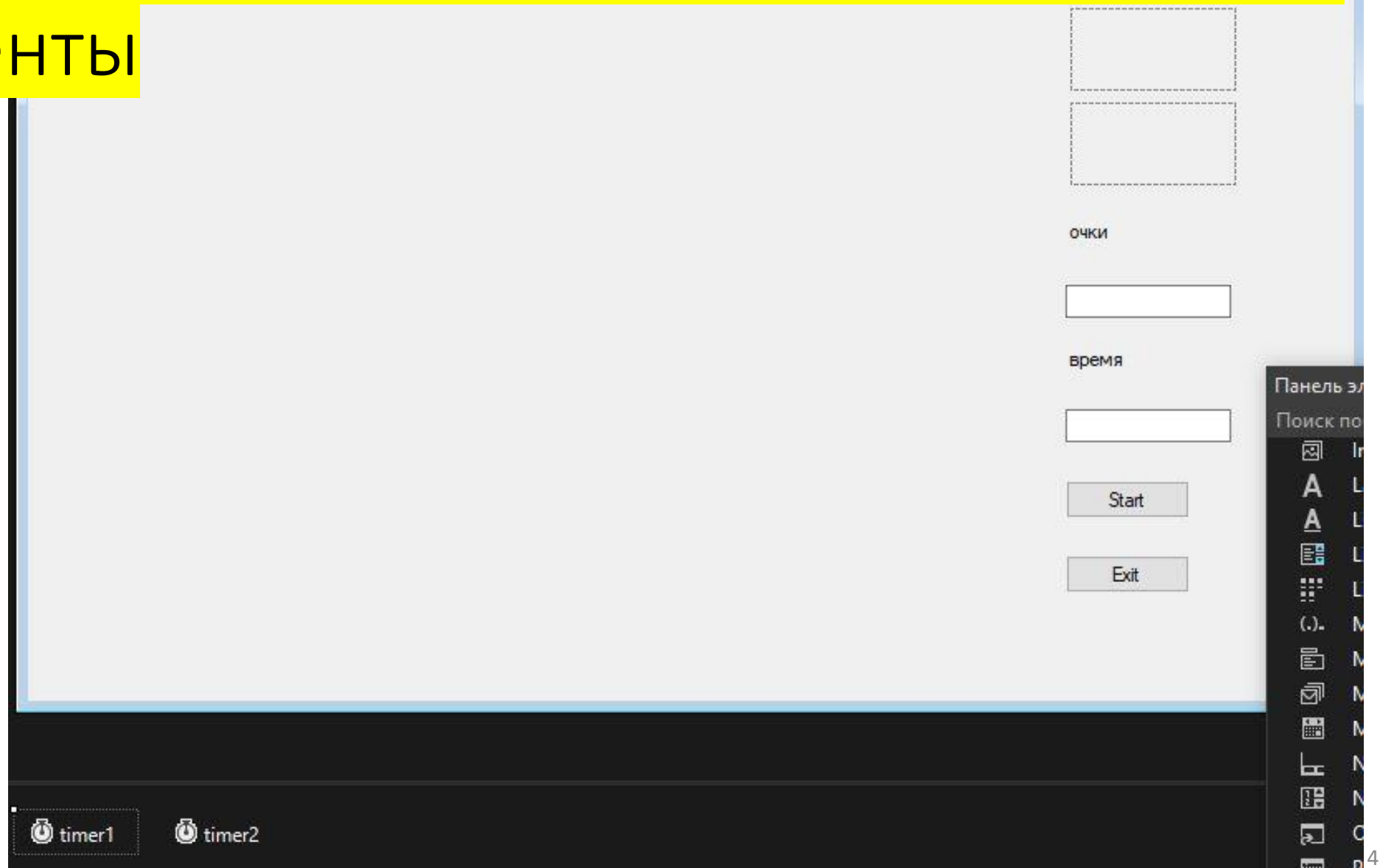
очки

время

Start

Exit

В конструкторе нужно создать следующие элементы



Глобальные переменные

- `public partial class Form1 : Form`
- `{`
- `Graphics g;`
- `public int[,] pole; //Двумерный массив pole.`
- `const int n = 4; //Кол-во строк и столбцов.`
- `string[] fn = new string[8] { "a1.JPG", "a2.JPG", "a3.JPG", "a4.JPG", "a5.JPG",`
`"a6.JPG", "a7.JPG", "a8.JPG" };`
- `Image[] zv = new Image[8]; //Массив рисунков`
- `PictureBox[,] Pb = new PictureBox[4, 4]; //Массив игры-сюда будем вставлять картинки`
- `int x0 = 40;`
- `int y0 = 20;`
- `int tim; //Времена таймеров`
- `int tim2;`
- `int tim3;`
- `public static int clik = 0;`

Глобальные переменные(продолжение)

- `Random rnd = new Random();`
- `int ochki = 0;`
- `Image[,] zvr = new Image[n, n];`//Резервный массив для восстановления игрового поля
- `int h, w;`// высота и ширина картинок
- `int ii1, jj1, ii2, jj2;`
- `int test = 0;`

Массив чисел $role[i,j]$ для расположения картинок

- 1 2 4 8
 - 1 3 4 5
 - 2 3 5 6
 - 7 7 6 8
-
- Числа от 1 до 8 располагаются случайно – две 1, две 2,
 - Две 3 и т. Д.

Массив чисел pole[i,j] для расположения картинок

```
• //-----  
• private void InitializeArray() //Инициализация двумерного массива pole.  
• {  
•     pole = new int[n, n]; //Инициализация двумерного массива pole.  
  
•     //Заполнение массива нулями.  
•     for (int i = 0; i < n; i++)  
•     {  
•         for (int j = 0; j < n; j++)  
•         {  
•             pole[i, j] = 0;  
•         }  
•     }  
• }  
• //-----
```

Массив чисел pole[i,j] для расположения картинок (продолжение)

- private void FillArray_2() {
- /*Задача 2. Заполните массив так:
- * в случайные места задайте следующие числа:
- * две единицы, две двойки, ..., две восьмерки.
- Random rand = new Random();
- int r = rand.Next(0, n);
- int k = rand.Next(0, n);
- int sum = 0;
- pole[r, k] = sum % 8
- for (int i = 0; i < n; i++)
- { for (int j = 0; j < n; j++)
- { sum++;
- while (pole[r, k] != 0) {
- r = rand.Next(0, n);
- k = rand.Next(0, n);
- }
- pole[r, k] = sum % 8;
- }}}

Иницилируем графику

- `public Form1()`
- `{`
- `InitializeComponent();`
- `g = this.CreateGraphics();`
- `}`

Показываем игровое поле

- `public void Pict_show()`
- `{//Показываем игровое поле`
- `int i, j;`
- `for (i = 0; i < 4; i++)`
- `for (j = 0; j < 4; j++)`
- `{`
- `Pb[i, j].Image = zvr[i,j];`
- `}`
- `}`
- `//zvr[i,j]` массив типа `Image`, в нем хранятся картинки
- `// Pb[i, j]` – массив типа `PictureBox`, в него вставляем картинки
- `//Эти массивы создаются в функции public void Create_pole()`, она показана ниже

Прячем изображения перед началом игры

- `public void Pict_hide()`
- `{//Прячем изображения`
- `int i, j;`
- `for (i = 0; i < 4; i++)`
- `for (j = 0; j < 4; j++)`
- `Pb[i, j].Image = null;`
- `}`

Создаем сетку, в ячейки которой вставляем картинки

- `public void Setka()`
- `{//Рисуем сетку`
- `int i;`
- `int w = 75;`
- `int h = 90;`
- `Pen myPenRed = new Pen(Color.Red, 3);`
- `for (i = 0; i < 5; i++)`
- `{`
- `g.DrawLine(myPenRed, ((x0 - 3) + i * (w + 10)), (y0 - 3), ((x0 - 3) + i * (w + 10)),`
`((y0 - 3) + 4 * (h + 10)));`
- `g.DrawLine(myPenRed, (x0 - 3), (y0 - 3 + i * (h + 10)), ((x0 - 3) + 4 * (w + 10)),`
`((y0 - 3) + i * (h + 10)));`
- `}`
- `}`

Функция для создания всех основных массивов и отрисовки поля игры

- `public void Create_pole()`
- `{//Игровое поле - массив PictureBox`
- `int i, j, k, m;`
- `w = 75;`
- `h = 90;`
-
- `Image[,] zv2 = new Image[4, 4];//Массивы для заполнения`
-
- `for (i = 0; i < 8; i++)//Массив рисунков`
- `zv[i] = new Bitmap("C:\\Users\\Administrator\\source\\repos\\ParyPict\\" + fn[i]);//вставляем`
`//рисунки в линейный массив`
- `for (i = 0; i < 4; i++)//Удаление прошлой игры`
- `for (j = 0; j < 4; j++)`
- `if (this.Controls.Contains(Pb[i, j]))`
- `this.Controls.Remove(Pb[i, j]);`

Функция для создания всех основных массивов и отрисовки поля игры(продолжение 1)

- for (i = 0; i < 4; i++)
- for (j = 0; j < 4; j++)
- {//Создаем массив игрового поля
- Pb[i, j] = new PictureBox();
- Pb[i, j].Left = x0 + j * (w + 10);
- Pb[i, j].Top = y0 + i * (h + 10);
- Pb[i, j].Width = w;
- Pb[i, j].Height = h;
-
- Pb[i, j].BorderStyle = BorderStyle.Fixed3D;
- Pb[i, j].Click += Pb_Click;
- Controls.Add(Pb[i, j]);
- }

Функция для создания всех основных массивов и отрисовки поля игры(продолжение 2)

- `InitializeArray();` //Инициализация двумерного массива `pole`.
- `FillArray_2();`//заполнение двумерного массива `pole` - числа для соответствия парным картинкам.
- `for (i = 0; i < n; i++)`
- `for (j = 0; j < n; j++)`
- `{`
- `zvr[i, j] = zv[pole[i, j]];`// заполнение резервного массива `zvr[i, j]`
- `Pb[i, j].Image = zvr[i, j];`//заполнение свойства `Image` у игрового поля
- `}`
- `}`
- `//конец функции public void Create_pole()`

Обработчик события Tick второго таймера. Он нужен, чтобы, если мы неправильно открыли парную картинку, могли на нее некоторое время посмотреть `tim3=5`. Переменная `test=1` означает, что картинку открыли неудачно

- `private void timer2_Tick(object sender, EventArgs e)`
- `{`
- `tim3++;`
- `if ((tim3==5)&&(test==1))`
- `{`
- `Pb[ii2, jj2].Image = null; //изображение спряталось`
- `Pb[ii1, jj1].Image = null; //изображение спряталось`
- `}`
- `}`

Обработчик события Tick первого таймера. Он нужен для того, чтобы, во-первых, мы могли некоторое время посмотреть на расположение картинок, прежде чем таймер их закроет, во вторых, этот таймер отсчитывает общее время игры tim2

```
• private void timer1_Tick(object sender, EventArgs e)
• {
•     tim = tim - 1; //Какое-то время видны все рисунки
•     if (tim == 0)
•     {
•         Pict_hide();
•     }
•     tim2--; //отсчет времени игры, доходит до 0 – игра останавливается
•     textBox2.Text = tim2.ToString();
•     if (tim2 < 0)
•     {
•         textBox2.Text = "Game Over!";
•     }
• }
```

Обработка click (щелчка мыши) по картинке

- `public void Pb_Click(object sender, EventArgs e)`
- `{//Обработчик щелчка по Pb[i,j]`
- `PictureBox Pib;`
- `Pib = (PictureBox)sender;`
- `if (tim2 > 0)`
- `{`
- `clik++;`
- `if (clik == 1)// нажали на первую картинку из пары`
- `{`
- `jj1 = (Pib.Left - x0) / w;//Номер столбца и строки элемента,`
- `ii1 = (Pib.Top - y0) / h;//по которому кликнули - запомнили.`
- `Pb[ii1, jj1].Image = zvr[ii1, jj1]; // открыли рисунок`
- `Pb[ii1, jj1].Enabled = true;`
- `}`

Обработка click (щелчка мыши) по картинке (продолжение 1)

- if (clik == 2) {// это если щелчок по предполагаемой паре
- jj2 = (Pib.Left - x0) / w;//Номер столбца и строки элемента,
- ii2 = (Pib.Top - y0) / h;//по которому кликнули - запомнили.
- Pb[ii2, jj2].Image = zvr[ii2, jj2];
- Pb[ii2, jj2].Enabled = true;
- if ((pole[ii1, jj1] == pole[ii2, jj2]) && (pole[ii1, jj1] != -1))
- {//Если угадали
- MessageBox.Show("Ok");
- ochki++;
- textBox1.Text = ochki.ToString();
- if (ochki == 8)
- { MessageBox.Show("Победа");
- pole[ii1, jj1] = -1;
- pole[ii2, jj2] = -1;
- test = 0;
- tim3 = 15; }

Обработка click (щелчка мыши) по картинке (продолжение 2)

- else { //открыли неправильную картинку
- tim3 = 0; // начинаем отсчет времени, когда на неправильну картинку еще можно поглядеть
- test = 1;
- }
- clik = 0; //в любом случае, переменная click зануляется
- }
- }
- }

Обработчик Click на кнопке начала игры

- private void button_start_Click(object sender, EventArgs e)
- {
- //Запуск игры
- Create_pole();
- Setka();
- tim = 10;
- tim2 = 1200;
- timer1.Enabled = true;
- timer2.Enabled = true;
- }

Обработчик Click на кнопке конца игры

- private void button_exit_Click(object sender, EventArgs e)
- {
- //Выход
- timer1.Enabled = false;
- timer2.Enabled = false;
- this.Close();
- }

Вроде и все.

- Если чего-то не хватает, смотрите текст программы